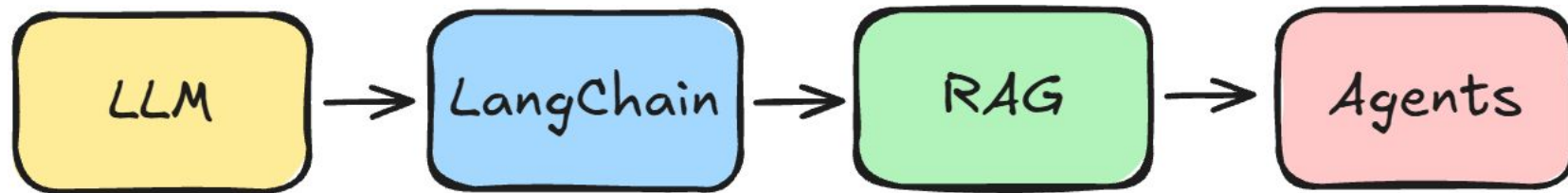


Лекція 24

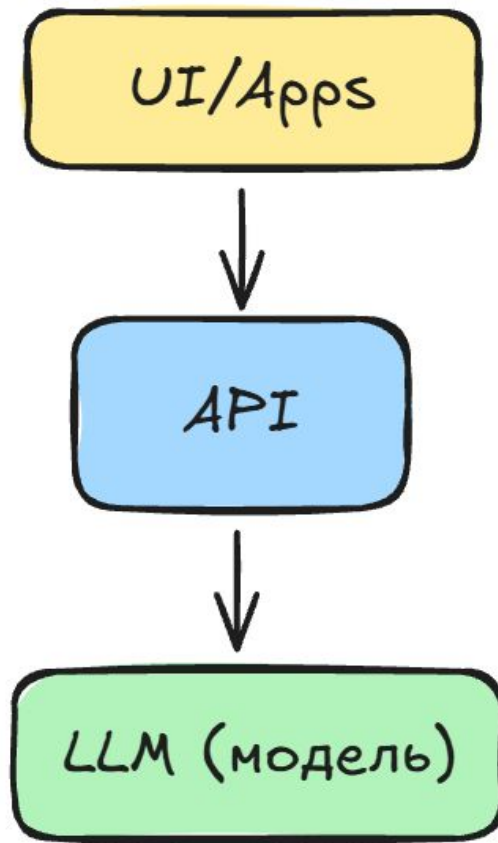
Основи розробки AI-асистентів. RAG



ChatGPT ≠ AI

ChatGPT, Gemini, Claude – це вебсайти або додатки.

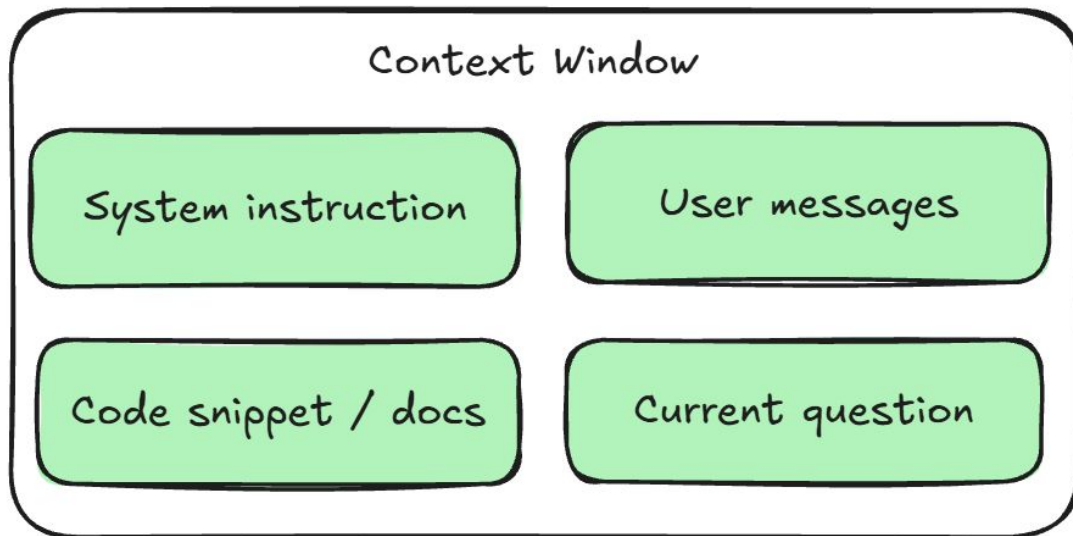
GPT-5.2, GPT-4o, Opus 4.5 – це LLM (Large Language Model).





LLM (Large Language Model) – це тип штучного інтелекту (ШІ), потужна нейронна мережа, навчена на величезних обсягах текстових даних для розуміння, обробки та генерації людської мови, що дозволяє їй виконувати різноманітні завдання, як-от написання текстів, переклад, підсумовування та відповіді на запитання, використовуючи складні закономірності мови для створення зв'язаних та контекстуально релевантних відповідей

Context window + Tokens



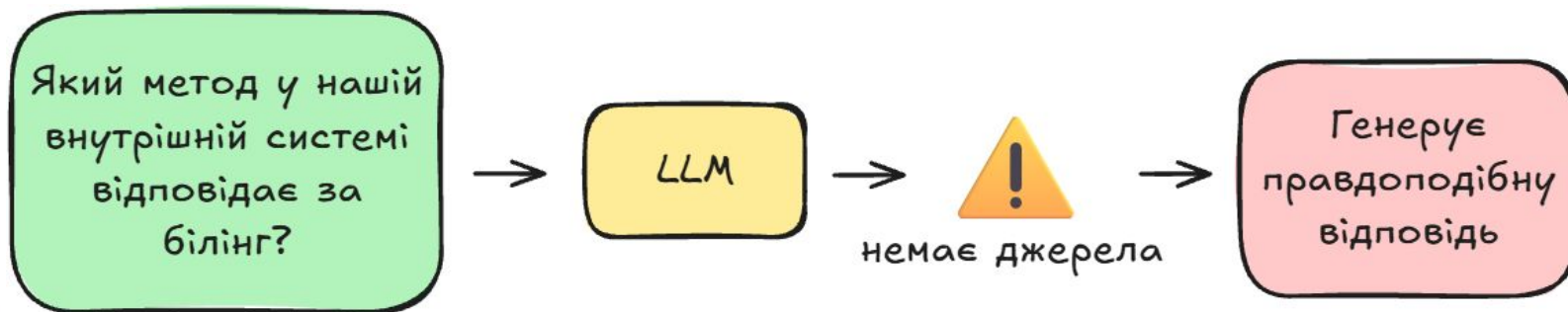
Token — це маленький шматок тексту, з яким реально працює мовна модель:

“hello” → [31245]

“найвірогідніший” → най + вір + огід + ніш + ий → [1542, 1987, 45, 9981, 123]

Hallucinations

Пам'ятайте: модель завжди намагається
відповісти



Tool calling

Tools:

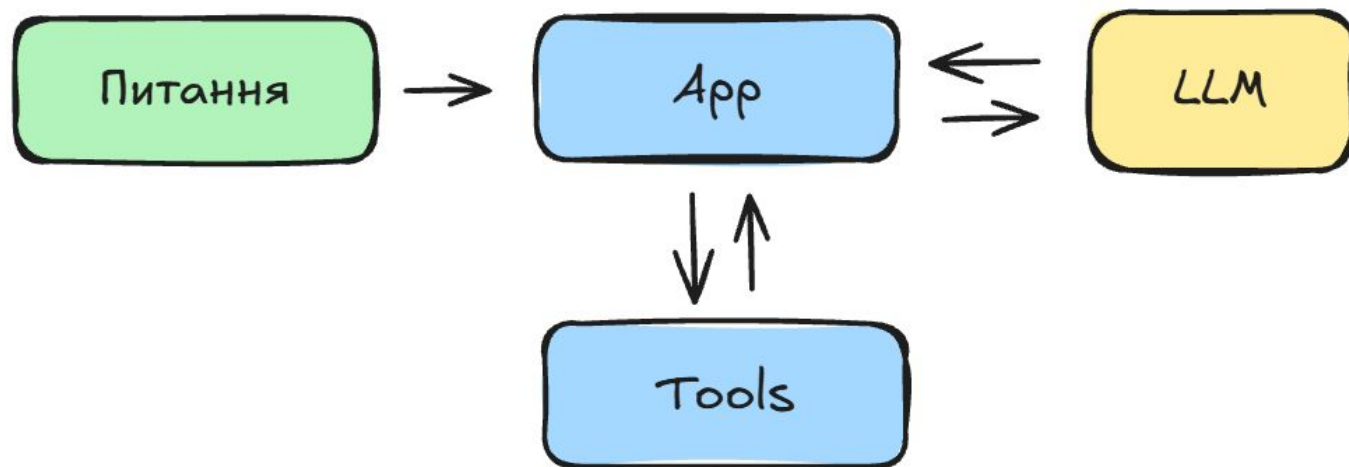
Web search

Calculator

DB query

Internal API

...



Cost/Latency компроміс (інженерна реальність)

Кожна взаємодія з моделлю має свою ціну

Cost

Latency

Quality

Приклад взаємодії з API

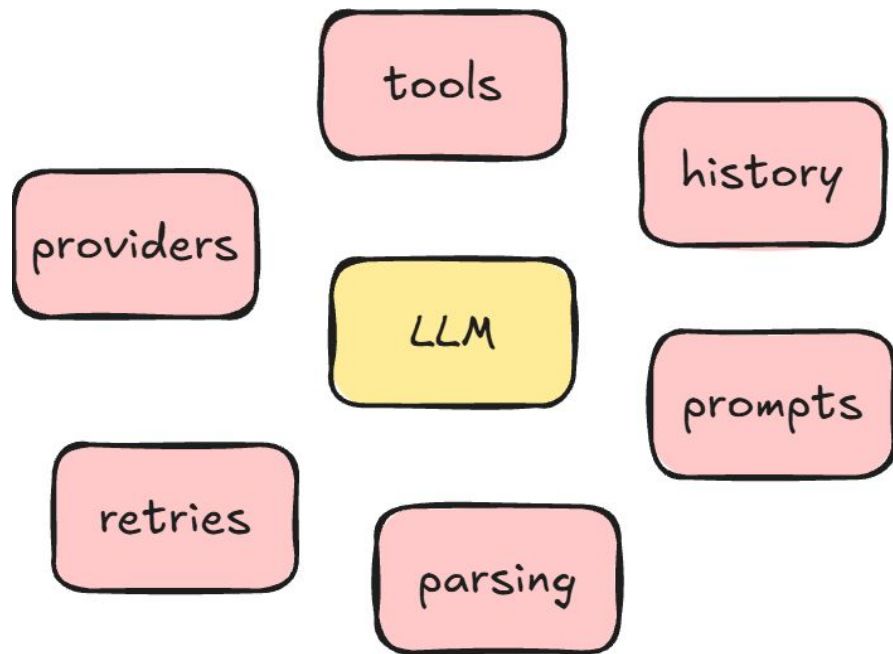
Для OpenAI:

1. Перейти на <https://platform.openai.com/api-keys>
2. Сформувати токен "Create new secret key"
3. НЕ показуй нікому свої токени ⚠

Ядро будь-якої LLM-системи

Усі LLM-системи, незалежно від складності, зводяться до цього ядра.





Production reality

Коли ми йдемо від демо до продукту,
складність з'являється не в моделі, а в обв'язці.

LangChain: шар оркестрації

LangChain не замінює LLM — він впорядковує роботу з нею.



Що дає LangChain на практиці



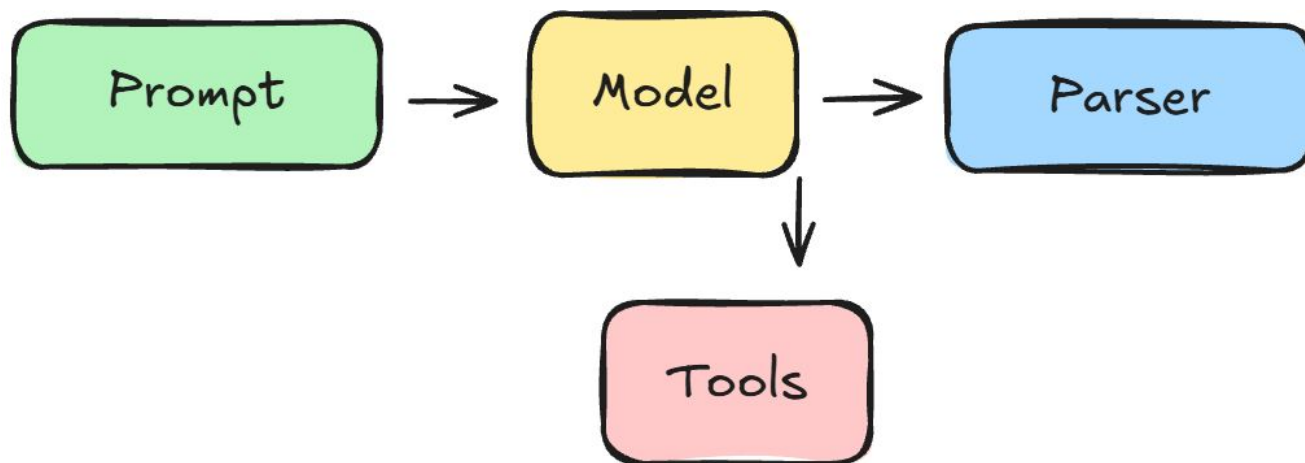
One interface — many models



Standard building blocks

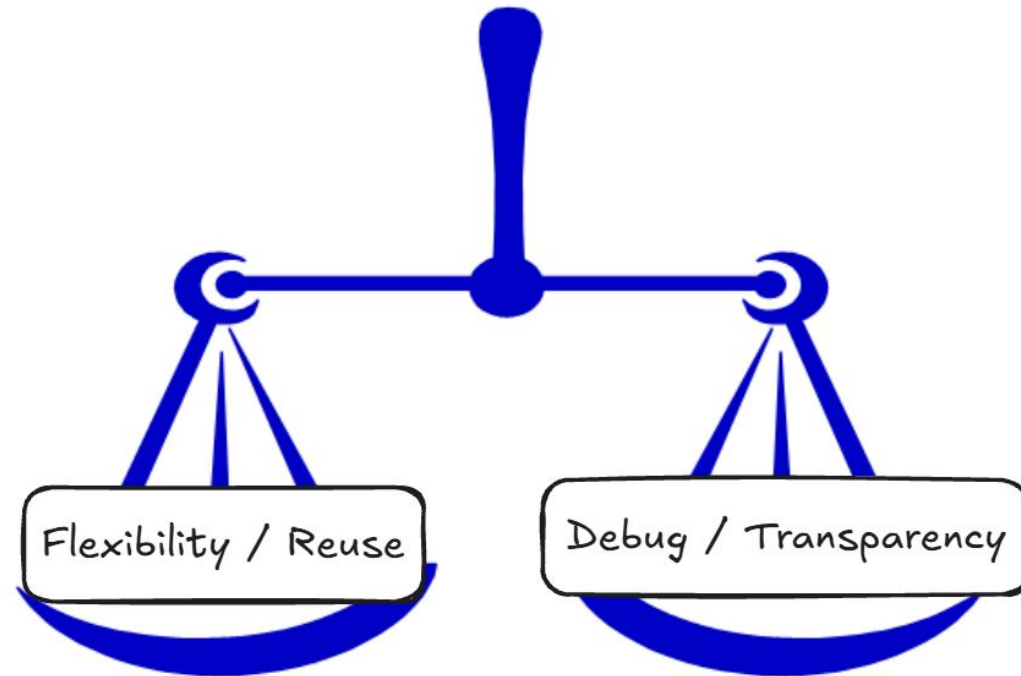


Ready for growth



Ціна абстракції

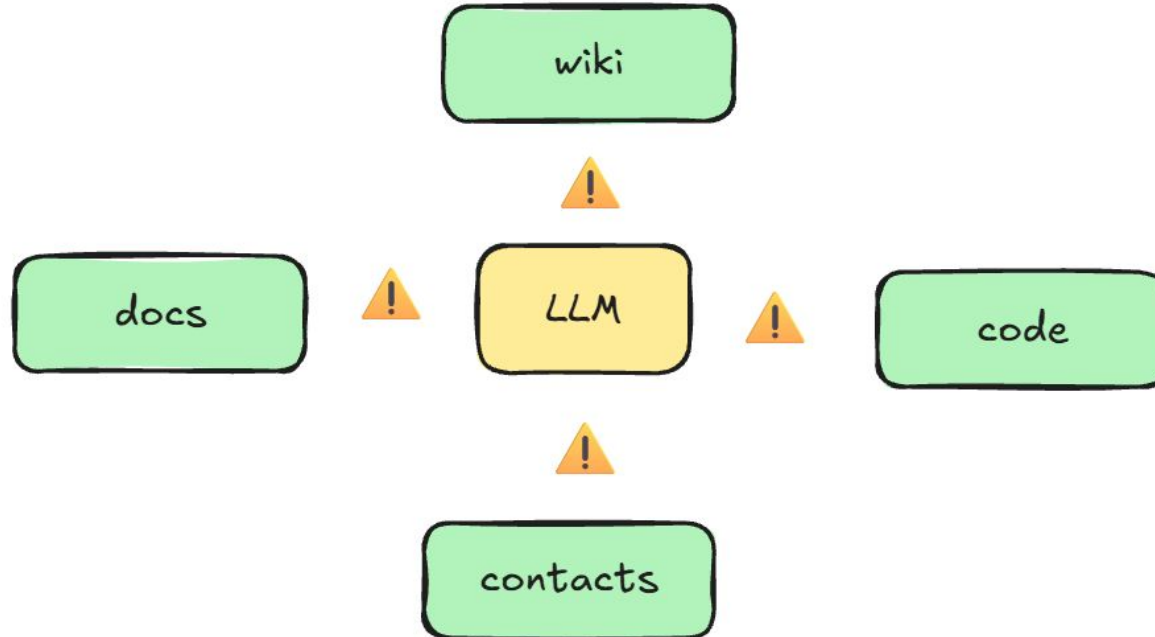
Кожен додатковий шар робить систему зручнішою — але менш прозорою.



Проблема LLM у реальних продуктах

LLM не має доступу до ваших даних

Працює лише з переданим контекстом



Чому “просто передати все” не працює



Обмежений context window



Дорого і

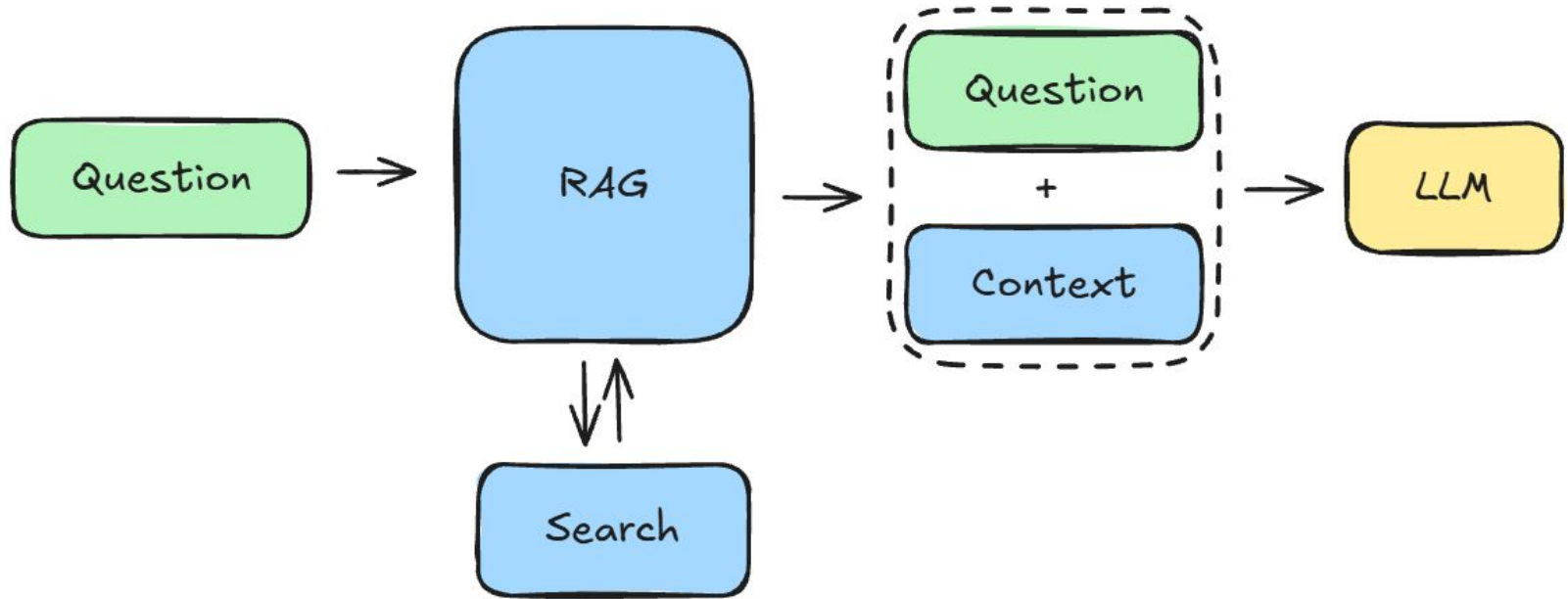


Повільно



Немає фокусу на релевантному

RAG – Retrieval Augmented Generation



- Знаходимо релевантні фрагменти
- Передаємо тільки їх
- Генеруємо відповідь на основі джерела

Ключова проблема — комп'ютер не розуміє сенс

Текст для комп'ютера = символи і рядки

Сенс він не розуміє

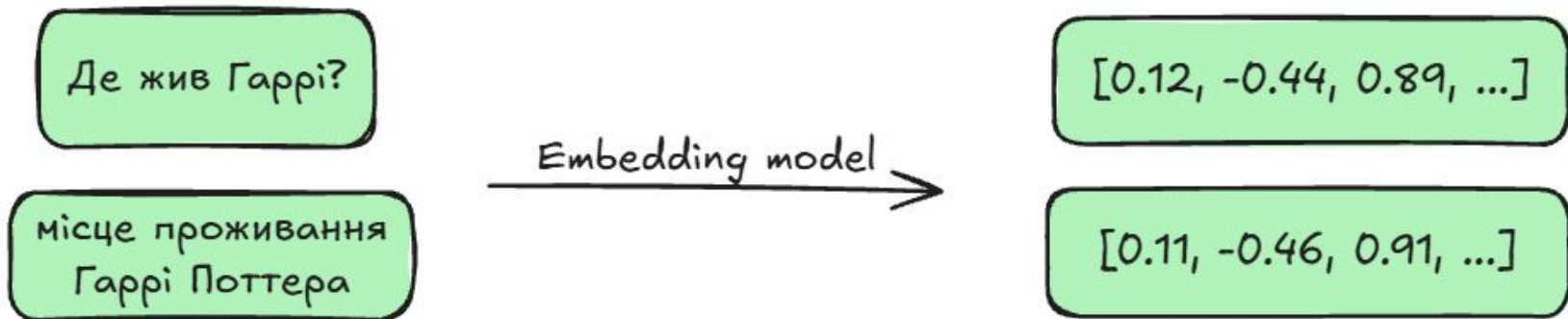
Пошук за словами $\neq$ пошук за змістом



Embeddings

Embedding — це числове представлення сенсу тексту.

Тексти з подібним змістом → схожі числові вектори.



RAG = підготовка + пошук

RAG — це не про LLM.

RAG — це про дані і пошук.

Крок 1
Підготовка

- Зчитуємо документи
- Розбиваємо на шматки
- конвертуємо в ембедінги
- зберігаємо у векторній БД

Крок 2
Пошук

- Конвертуємо питання в ембедінг
- виконуємо пошук к векторній БД

Агент — це патерн використання LLM, у якому модель:

- аналізує задачу
- вирішує, що потрібно зробити далі
- використовує доступні інструменти
- аналізує результат
- і повторює цей процес, доки не отримає фінальну відповідь

Агент = LLM + ваш код

Агент — це завжди співпраця

Модель приймає рішення

Програма:

- виконує інструменти
- контролює цикл
- повертає результат назад

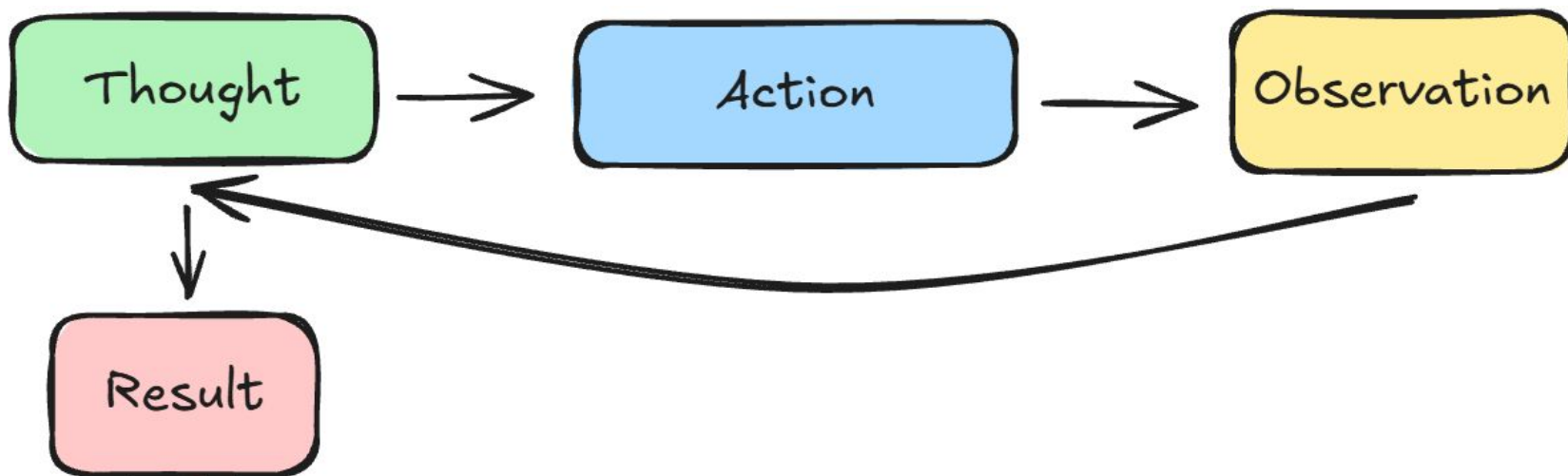
LLM (thinking)

Code(doin)

Основний цикл агента (ReAct)

Майже всі агенти працюють однаково:

- Аналіз
- Вибір дії
- Отримання результату
- Повтор або відповідь



Інструменти агента

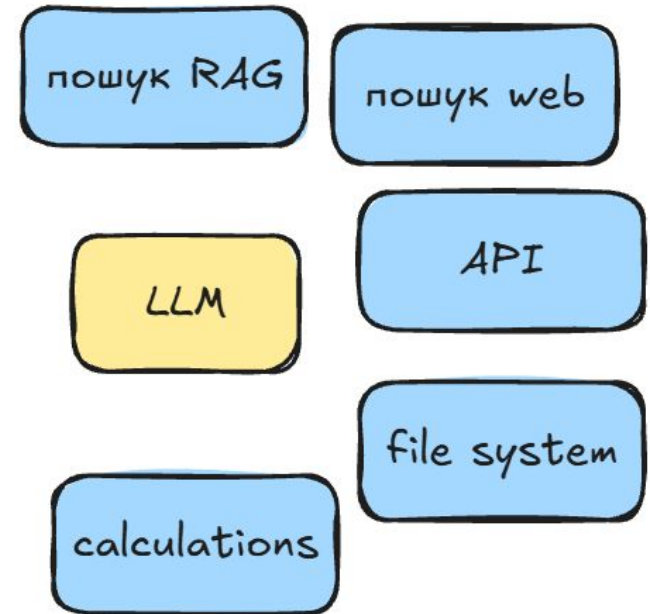
Агент сам нічого не вміє

Усі можливості — це інструменти

Типові приклади:

- пошук (RAG, web)
- API
- обчислення
- робота з файлами

Для моделі вони виглядають однаково



Demo

<https://github.com/ZablotskyiMichael/llm-demo>

<https://github.com/ZablotskyiMichael/llm-chat>

<https://github.com/ZablotskyiMichael/llm-langchain>